

Prática DevOps: Containerização Social Ledger

Disciplina: DevOps

Autor: João Paulo Morais Rangel

1. Visão Geral da Aplicação

O **Social Ledger** é uma ferramenta full-stack de simulação financeira projetada para simplificar a gestão de despesas compartilhadas entre grupos de pessoas. A aplicação permite que os usuários criem grupos, registrem participantes (incluindo convidados simulados), lancem gastos e visualizem o fechamento de contas.

O diferencial técnico da ferramenta é o seu algoritmo de liquidação de dívidas implementado no backend. Utilizando uma abordagem de grafos com *Priority Queues* (Filas de Prioridade), o sistema calcula o número mínimo de transações necessárias para zerar todos os débitos do grupo, resolvendo de forma eficiente o problema de compensação "N-para-N".

2. Tecnologias Utilizadas

Camada	Tecnologia	Descrição / Papel
Backend	Java 21 / Spring Boot 4	API RESTful com arquitetura limpa (Clean Architecture).
Frontend	React / TypeScript / Vite	Interface Single Page Application (SPA) responsiva.
Banco de Dados	PostgreSQL 15	Persistência de dados relacional e transacional.
Servidor Web	Nginx	

Camada	Tecnologia	Descrição / Papel
		Hospedagem do frontend e Proxy Reverso para a API.
Orquestração	Docker & Docker Compose	Gerenciamento do ciclo de vida dos contêineres e redes.

3. Arquitetura de Contêineres (DevOps)

Para atender aos requisitos da disciplina, a aplicação foi estruturada em três serviços independentes que se comunicam através de uma rede interna isolada:

3.1. Contêiner de Banco de Dados (db)

- **Imagem:** `postgres:15-alpine`.
- **Persistência:** Utiliza um volume nomeado (`postgres_data`) para garantir que os dados não sejam perdidos ao reiniciar os contêineres.

3.2. Contêiner de API (api)

- **Construção:** Utiliza *Multi-stage build*. O primeiro estágio usa Maven para compilar o código Java 21, e o segundo utiliza um JRE leve para execução.
- **Conectividade:** Conecta-se ao banco de dados via rede interna usando o DNS do Docker (`db:5432`).

3.3. Contêiner de Frontend (frontend)

- **Papel:** Atua como servidor web de arquivos estáticos e **Proxy Reverso**.
- **Configuração:** O Nginx intercepta requisições enviadas para `/api` e as redireciona internamente para o contêiner `api`. Isso elimina problemas de CORS no navegador e centraliza o acesso na porta 80.

4. Manual de Instalação e Execução

Graças à containerização, a aplicação pode ser executada em qualquer ambiente que possua o Docker instalado, sem a necessidade de configurar Java, Node.js ou PostgreSQL localmente.

Pré-requisitos

- Docker Desktop ou Docker Engine instalado.
- Docker Compose configurado.

Passos para Execução

1. Navegue até a raiz do projeto onde se encontra o arquivo `docker-compose.yml`.
2. Execute o comando para construir e iniciar os serviços:

```
docker compose up -d --build
```

3. Aguarde a inicialização dos serviços (o banco de dados e a API levam alguns segundos para estarem prontos).
4. Acesse a aplicação no navegador em: <http://localhost>

5. Roteiro de Testes (Passo a Passo)

Para guiar a avaliação da aplicação e garantir a validação de todas as camadas do sistema, siga o roteiro estruturado abaixo:

1. **Autenticação Inicial:** Na tela de login, utilize as credenciais padrão geradas automaticamente pelo mecanismo de inicialização de dados (*Data Seed*):
 - **E-mail:** `guest@email.com`
 - **Senha:** `guest1234`
2. **Cadastro de Participantes:** Navegue até a aba ou seção de **Groups**. Antes de criar um grupo, utilize o botão ou formulário de **Register** para cadastrar algumas pessoas (por exemplo, adicione 3 ou 4 amigos ou integrantes fictícios). Isso alimentará a base de dados simulada.

3. **Criação do Grupo de Simulação:** Ainda na área de gerenciamento, crie um novo grupo de simulação financeira. Dê um nome ao grupo (ex: "Viagem de Férias" ou "Dividindo o Aluguel") e selecione os participantes que você acabou de registrar para comporem este grupo. Salve as alterações.
4. **Acesso ao Painel Central:** Dirija-se à aba **Dashboard**. No seletor de contexto da interface, escolha o grupo recém-criado. A tela atualizará exibindo o saldo de cada membro zerado e o gráfico de transações vazio.
5. **Inclusão e Divisão de Despesas:** Clique no botão para adicionar uma nova transação. Preencha uma descrição (ex: "Jantar de Boas-Vindas") e um valor total (ex: 300.00). Selecione o participante que pagou inicialmente essa conta inteira.
6. **Ajuste de Balanço (Etapa Crítica):** Selecione quais membros participaram desse gasto. **Atenção:** Certifique-se de clicar explicitamente na opção **Split Equally** (Dividir Igualmente) para que o sistema distribua os pesos de forma uniforme entre os selecionados. Essa ação valida a consistência contábil da transação na interface; caso contrário, o botão de salvar permanecerá desabilitado para evitar balanços inconsistentes no backend. Salve a transação.
7. **Validação do Algoritmo de Grafos:** Crie mais uma ou duas transações alternando quem pagou e quem divide. O Dashboard atualizará em tempo real, exibindo os balanços líquidos de cada indivíduo e, na seção de liquidação, o resultado do algoritmo baseado em *Priority Queues*, indicando exatamente quem deve pagar para quem, reduzindo o fluxo de transações ao menor número físico possível.

Nota de Manutenção de Ambiente: Para reiniciar o ambiente de testes limpando completamente o banco de dados e os registros gerados, utilize o comando `docker compose down -v` no terminal.

6. Comandos Úteis

- Verificar logs em tempo real: `docker compose logs -f`
- Parar aplicação preservando os dados: `docker compose down`
- Parar aplicação removendo volumes (limpeza total): `docker compose down -v`

Prática DevOps: Containerização Social Ledger - UFSCar

2026 - Projeto Social Ledger